MINGGU KE-6 • PEMROGRAMAN DAN PENGUJIAN APLIKASI WEB (12S3101)

Database Connectivity pada Laravel 11: Arsitektur, Teori Dasar, & Studi Kasus

Dari Konsep Koneksi TCP/PDO, Konfigurasi Environment, Migration, Query Builder, Eloquent ORM, hingga Transaksi ACID

Dosen Pengampu: Chandro Pardede, S.Kom., M.Sc. • **Pemrograman dan Pengujian Aplikasi Web (12S3101)**
Institut Teknologi Del • FITE • Program Studi S1 Sistem Informasi

CAPAIAN PEMBELAJARAN (OBJECTIVES) MINGGU KE-6

POIN PEMBAHASAN UTAMA:

- Menjelaskan konsep dasar konektivitas basis data pada aplikasi web
- Memahami arsitektur komunikasi TCP/IP, siklus koneksi, dan driver PDO
- Mengonfigurasi koneksi database melalui .env dan config/database.php
- Menguasai struktur Migration, Seeder, Query Builder, dan Eloquent ORM
- Mengimplementasikan aplikasi CRUD dengan transaksi ACID di Laravel 11

KORELASI DENGAN MATERI MINGGU 5 (MVC):

Di Minggu 5 kita mempelajari peran 'Model'. Minggu 6 ini membedah bagaimana Model berkomunikasi langsung dengan mesin RDBMS.

Peta Kompetensi Minggu 6:

```
[Minggu 5: Arsitektur MVC Laravel]
        |
        ▼
[Minggu 6: Database Connectivity]
 ├── 1. Teori Koneksi (TCP, PDO, Binding)
 ├── 2. Topik Konektivitas (Env, ACID)
 ├── 3. Struktur DB Laravel (ORM, Schema)
 └── 4. Studi Kasus Nyata (SIAKAD KRS)
```

PETA ALUR PEMBAHASAN (ROADMAP) MATERI

POIN PEMBAHASAN UTAMA:

- 1. Objective & Latar Belakang: Kebutuhan persistence storage di web
- 2. Teori Dasar Connectivity: Socket TCP/IP, PDO, DSN, Parameter Binding
- 3. Topik Utama Connectivity: Multi-DB, Read/Write Split, & Transaksi
- 4. Struktur Database Laravel 11: Folder config, migrations, & Models
- 5. Studi Kasus & Pengerjaan: Sistem KRS & Mata Kuliah langkah demi langkah

METODOLOGI PEMBELAJARAN:

Setiap konsep teoritis langsung dipasangkan dengan contoh kode Laravel 11 agar mudah dipahami dan siap dipraktikkan.

Sistematika 36 Halaman Slide:

```
Bagian 1 (01-03) : Objective & Roadmap
Bagian 2 (04-06) : Latar Belakang Masalah
Bagian 3 (07-12) : Teori Dasar Connectivity
Bagian 4 (13-19) : Topik Bahasan Connectivity
Bagian 5 (20-26) : Struktur DB di Laravel 11
Bagian 6 (27-34) : Contoh Kasus & Pengerjaan
Bagian 7 (35-36) : Best Practices & Evaluasi
```

LATAR BELAKANG: MEMORI EFEMERAL VS PERSISTENCE

POIN PEMBAHASAN UTAMA:

- Eksekusi skrip PHP bersifat stateless dan hidup hanya selama 1 request
- Variabel di RAM langsung terhapus saat HTTP response selesai dikirim
- Aplikasi membutuhkan Persistence Storage agar data tersimpan permanen
- RDBMS (MySQL, PostgreSQL) menyimpan data di disk secara terstruktur
- Database Connectivity menjadi jembatan antara aplikasi dan server DBMS

MENGAPA PERLU DATABASE CONNECTIVITY?

Tanpa konektivitas basis data, data transaksi atau pendaftaran mahasiswa akan hilang seketika saat halaman web di-refresh.

Memori RAM vs Database Persisten:

```
// 1. Variabel RAM (Hilang setelah request)
$mhs = ["nim" => "12S21001", "ipk" => 3.85];
// Saat Response selesai -> $mhs MUSNAH!

// 2. Simpan Persisten ke MySQL via Laravel
Student::create([
    'nim' => '12S21001',
    'ipk' => 3.85,
]);
// Data aman permanen di Disk Database!
```

MASALAH KONEKSI DATABASE PADA PHP TRADISIONAL

POIN PEMBAHASAN UTAMA:

- Hardcoded Credentials: Password DB tersebar di banyak file skrip PHP
- Connection Leak: Lupa menutup koneksi memicu error Too Many Connections
- SQL Injection: Penggabungan string input langsung ke SQL sangat rawan
- Vendor Lock-in: Fungsi mysqli_* mengikat kode pada satu jenis DBMS saja
- Tanpa Version Control: Struktur tabel diubah manual tanpa riwayat tim

RISIKO KEAMANAN KODE PROSEDURAL:

Mayoritas insiden kebocoran data pada web tradisional berawal dari penulisan koneksi dan query SQL mentah yang tidak terstandarisasi.

Contoh Buruk Koneksi PHP Native:

```php
<?php
// BURUK: Password hardcoded di dalam kode!
$db = mysqli_connect("localhost", "root", "123");

// BAHAYA: Input langsung digabung ke SQL!
$nim = $_GET['nim'];
$sql = "SELECT * FROM users WHERE nim='$nim'";

// Jika $nim berisi: ' OR '1'='1
// Seluruh data tabel bocor (SQL Injection)!
$res = mysqli_query($db, $sql);
```

KEUNGGULAN ARSITEKTUR DATABASE PADA LARAVEL 11

POIN PEMBAHASAN UTAMA:

- Sentralisasi kredensial secara aman melalui file environment (.env)
- Lazy Connection: Koneksi TCP hanya dibuka saat query benar-benar jalan
- Keamanan Otomatis: Seluruh query dilindungi PDO Prepared Statements
- Database Agnostic: Mudah beralih antara MySQL, PostgreSQL, & SQLite
- Ekosistem Lengkap: Migration, Seeder, Query Builder, & Eloquent ORM

STANDAR INDUSTRI MODERN:

Laravel mengabstraksi kerumitan socket jaringan dan driver PDO sehingga developer dapat fokus penuh pada logika bisnis aplikasi.

Koneksi Aman & Bersih di Laravel 11:

```
use App\Models\User;

// Ambil input dari HTTP Request
$nim = request('nim');

// AMAN: Otomatis menggunakan PDO Binding!
$user = User::where('nim', $nim)->first();

// Koneksi otomatis dibuka saat dibutuhkan
// dan ditutup saat siklus request selesai.
```

ARSITEKTUR CLIENT-SERVER & SOCKET TCP/IP

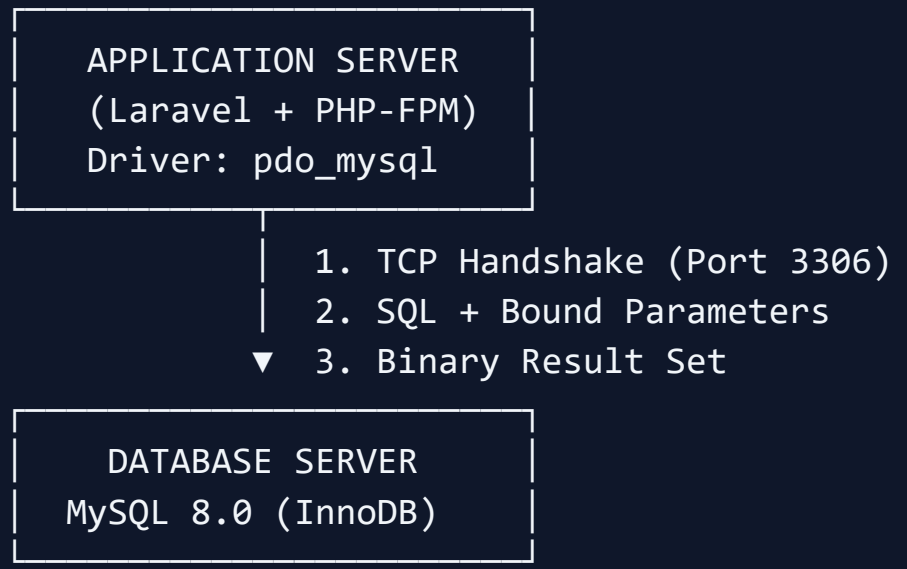
POIN PEMBAHASAN UTAMA:

- Aplikasi Laravel bertindak sebagai Database Client yang meminta data
- Mesin RDBMS (MySQL/PostgreSQL) bertindak sebagai Database Server
- Komunikasi jaringan menggunakan protokol TCP/IP (Port MySQL: 3306)
- Untuk server di mesin yang sama, dapat memakai Unix Domain Socket
- Pertukaran query dan hasil dikemas dalam paket biner protokol DBMS

PEMISAHAN SERVER DI CLOUD:

Pada arsitektur produksi, Application Server dan Database Server dipisahkan pada mesin berbeda demi keamanan dan skalabilitas.

Topologi Komunikasi Client-Server:



LIMA TAHAP SIKLUS HIDUP KONEKSI (LIFECYCLE)

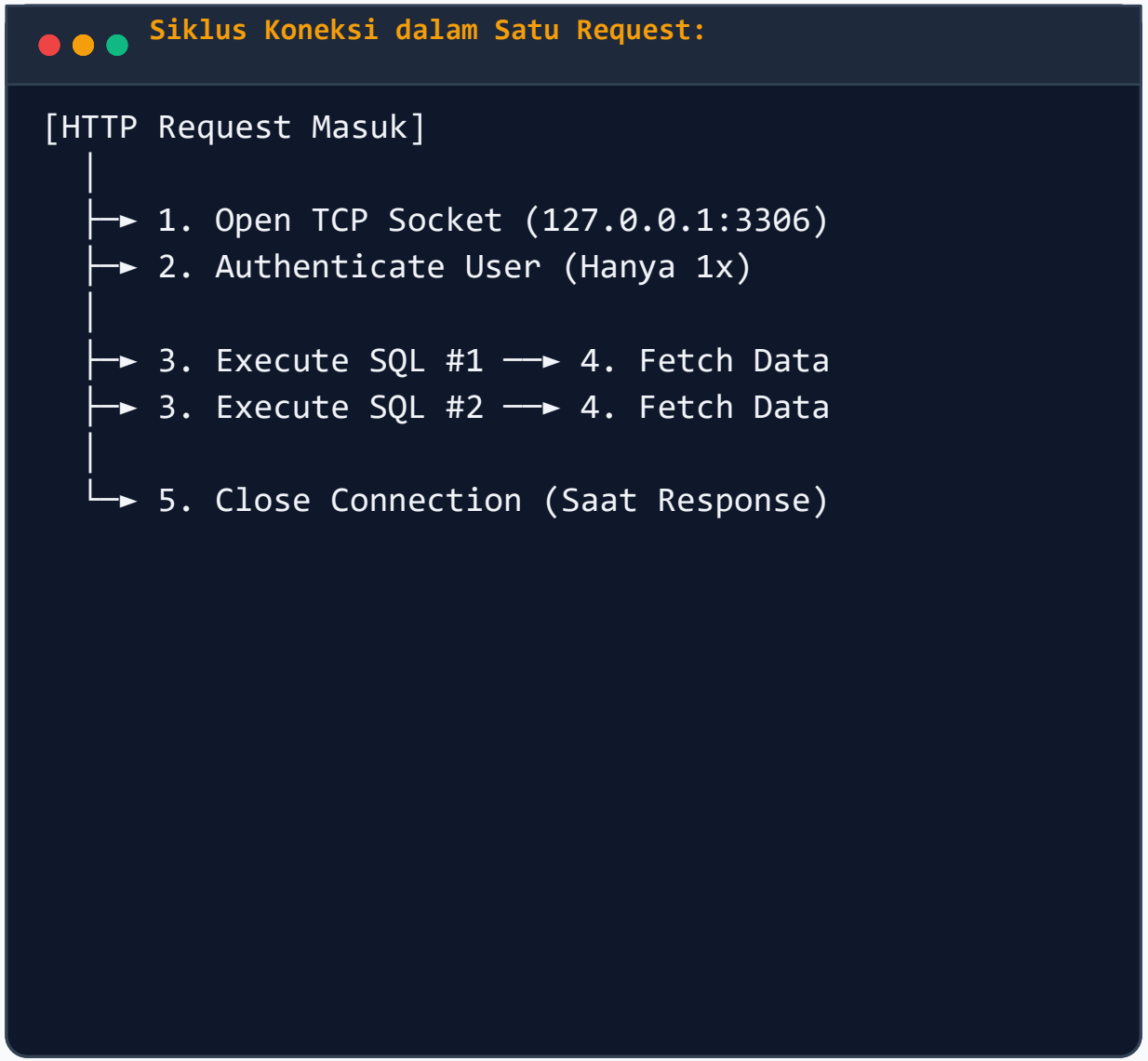
POIN PEMBAHASAN UTAMA:

- 1. Connection Open: Membuka kanal socket TCP ke alamat host & port DB
- 2. Authentication: Verifikasi username, password, dan hak akses skema
- 3. Query Execution: Mengirim statement SQL beserta parameter terikat
- 4. Result Fetching: Mengambil baris hasil query ke dalam memori PHP
- 5. Connection Teardown: Menutup koneksi di akhir siklus HTTP request

OPTIMASI SINGLETON CONNECTION:

Tahap 1 dan 2 memakan waktu paling lama (~10ms). Laravel membuka koneksi cukup 1 kali dan memakainya ulang untuk semua query.

Siklus Koneksi dalam Satu Request:



LAPISAN ABSTRAKSI PDO (PHP DATA OBJECTS)

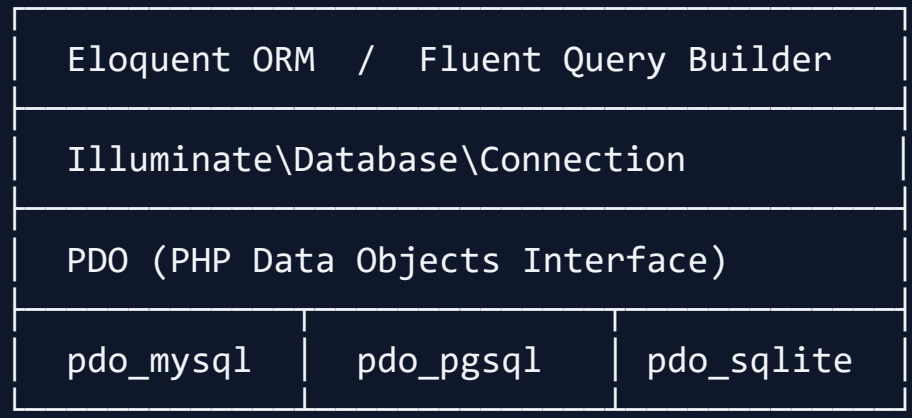
POIN PEMBAHASAN UTAMA:

- PDO adalah ekstensi C bawaan PHP yang menjadi fondasi DB di Laravel
- Menyediakan Data-Access Abstraction Layer yang seragam untuk semua DB
- Inti PDO bekerja sama dengan driver spesifik (pdo_mysql, pdo_pgsql)
- Mendukung penuh fitur Prepared Statements, Transactions, & Exception
- Memungkinkan pergantian vendor DBMS tanpa mengubah kode logika PHP

SYARAT KONFIGURASI PHP.INI:

Saat men-deploy aplikasi Laravel ke server, pastikan ekstensi php-pdo dan php-mysql sudah berstatus aktif (enabled) di php.ini.

Arsitektur Berlapis Database Laravel:



ANATOMI DATA SOURCE NAME (DSN) & OPSI PDO

POIN PEMBAHASAN UTAMA:

- DSN adalah string terformat untuk menginisialisasi koneksi objek PDO
- Memuat komponen: driver prefix (mysql:), host, port, dbname, & charset
- ERRMODE_EXCEPTION memaksa PDO melempar Exception saat query gagal
- ATTR_DEFAULT_FETCH_MODE mengatur format hasil baca (Object / Array)
- ATTR_EMULATE_PREPARES = false mengaktifkan native prepared statement

PERAN KELAS CONNECTOR LARAVEL:

Kelas MySqlConnector di Laravel bertugas merakit string DSN dan menyetel atribut PDO teraman secara otomatis di balik layar.

Inisialisasi PDO di Balik Layar Laravel:

```
// 1. String DSN dirakit oleh Connector:
$dsn = "mysql:host=127.0.0.1;port=3306;"
     . "dbname=siakad_del;charset=utf8mb4";

// 2. Opsi Keamanan Standar PDO Laravel:
$opts = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_EMULATE_PREPARES => false,
];

$pdo = new PDO($dsn, "root", "secret", $opts);
```

PREPARED STATEMENTS & PARAMETER BINDING

POIN PEMBAHASAN UTAMA:

- Mekanisme utama untuk menangkal serangan SQL Injection secara mutlak
- Memisahkan pengiriman struktur perintah SQL dari nilai input pengguna
- Tahap 1 (Prepare): DB mengkompilasi template SQL dengan placeholder (?)
- Tahap 2 (Execute): Data dikirim terpisah sebagai nilai literal murni
- Input berbahaya dari user tidak akan pernah dieksekusi sebagai komando

MENGAPA KEBAL SQL INJECTION?

Karena pohon sintaks SQL sudah dikunci pada Tahap 1, tanda kutip atau perintah berbahaya di dalam input hanya dibaca sebagai teks biasa.

Protokol Dua Fase Prepared Statement:

```
-- FASE 1: Server DB kompilasi struktur SQL
PREPARE stmt FROM
  'SELECT * FROM courses WHERE code = ?';

-- FASE 2: Kirim nilai parameter terpisah
SET @kode = '12S3101';
EXECUTE stmt USING @kode;

-- Otomatis dilakukan oleh Laravel saat:
Course::where('code', '12S3101')->first();
```

PRINSIP TRANSAKSI BASIS DATA (ACID PROPERTIES)

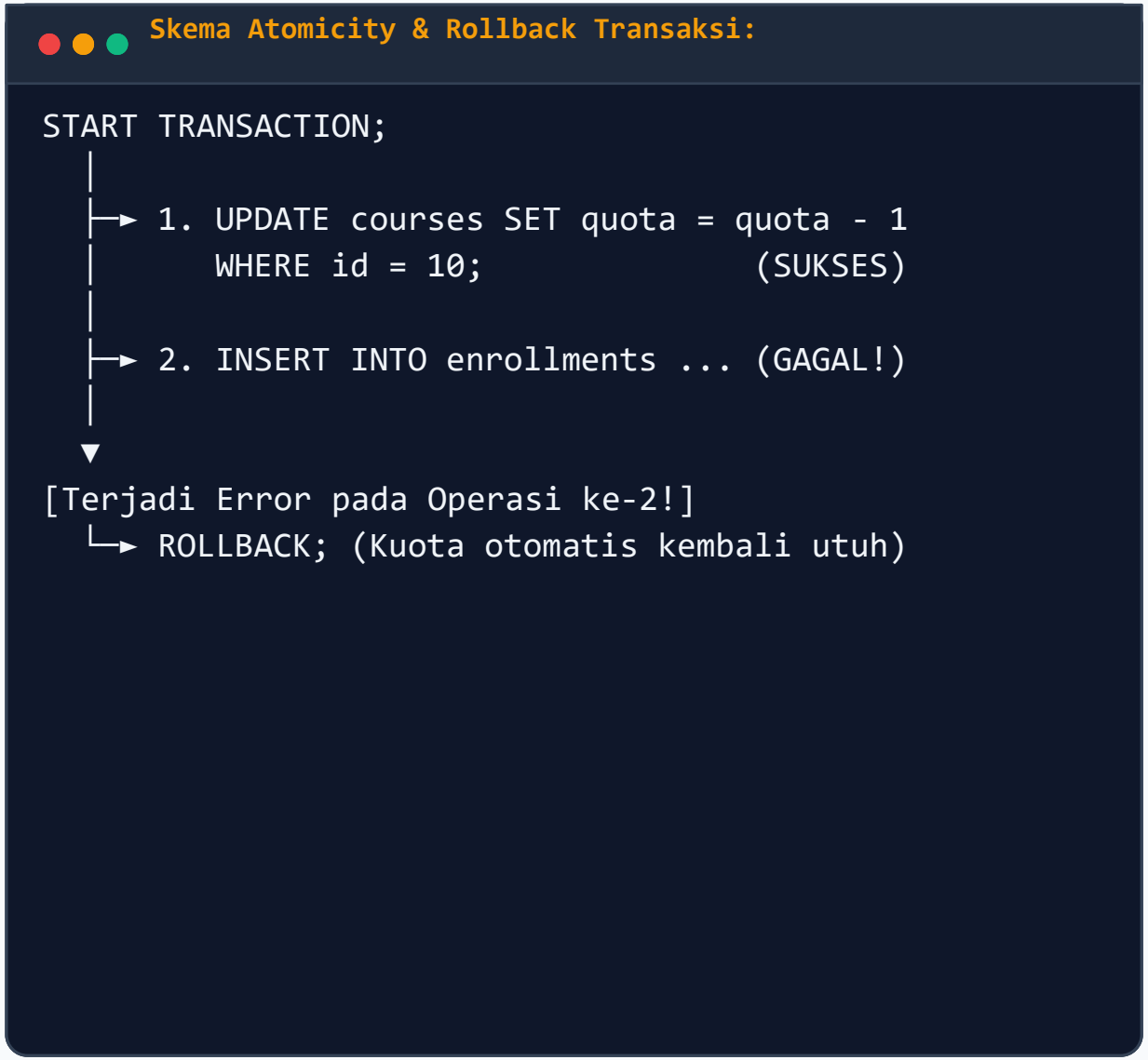
POIN PEMBAHASAN UTAMA:

- Atomicity: Seluruh operasi dalam transaksi sukses utuh atau batal semua
- Consistency: Transaksi menjaga keabsahan relasi dan constraint tabel
- Isolation: Eksekusi transaksi paralel tidak saling mengacaukan data
- Durability: Data yang sudah di-COMMIT tersimpan permanen di media disk
- Wajib diterapkan pada sistem keuangan, stok barang, dan kuota KRS

STORAGE ENGINE INNODB:

Laravel secara default menggunakan engine InnoDB pada MySQL karena mendukung penuh transaksi ACID dan Foreign Key.

Skema Atomicity & Rollback Transaksi:



TIGA CARA INTERAKSI DATABASE DI LARAVEL 11

POIN PEMBAHASAN UTAMA:

- 1. Raw SQL (DB::select): Menulis SQL murni untuk query laporan rumit
- 2. Query Builder (DB::table): Method chaining cepat untuk query & join
- 3. Eloquent ORM (Model): Memetakan tabel menjadi objek Class OOP penuh
- Ketiganya berjalan di atas koneksi PDO & parameter binding yang aman
- Praktik Terbaik: Gunakan Eloquent untuk CRUD, Query Builder untuk rekap

FLEKSIBILITAS ARSITEKTUR LARAVEL:

Developer bebas memilih pendekatan yang paling sesuai untuk tiap fitur tanpa kehilangan perlindungan keamanan PDO binding.

Tiga Pendekatan Query di Laravel 11:

```
// 1. Raw SQL dengan Parameter Binding
$a = DB::select(
    'SELECT * FROM courses WHERE credits = ?',
    [3]
);

// 2. Fluent Query Builder (stdClass)
$b = DB::table('courses')
       ->where('credits', 3)->get();

// 3. Eloquent ORM (Koleksi Objek Model)
$c = Course::where('credits', 3)->get();
```

MANAJEMEN KONFIGURASI ENVIRONMENT (.ENV)

POIN PEMBAHASAN UTAMA:

- Memisahkan konfigurasi kredensial dari kode program (12-Factor App)
- File .env menyimpan host, port, nama database, username, dan password
- Fungsi env() menyalurkan nilai dari .env ke dalam config/database.php
- File .env dilarang di-commit ke Git agar password tidak bocor ke publik
- Memudahkan pergantian DB antara Local, Staging, dan Server Production

DEFAULT DATABASE LARAVEL 11:

Laravel 11 menggunakan SQLite secara default agar mudah diinisialisasi, namun sangat mudah diubah ke MySQL/PostgreSQL lewat .env.

Parameter Koneksi pada File .env:

```
# File: .env (Rahasia & Spesifik Mesin)
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=siakad_del
DB_USERNAME=del_admin
DB_PASSWORD=RahasiaKampus2026!

# Atau menggunakan URL Koneksi Tunggal:
# DB_URL="mysql://root:pass@127.0.0.1/siakad"
```

PERSISTENT CONNECTIONS & CONNECTION POOLING

POIN PEMBAHASAN UTAMA:

- Normalnya, koneksi TCP ke database ditutup setiap kali request selesai
- Pada trafik tinggi, handshake TCP berulang kali membebani CPU database
- Opsi PDO::ATTR_PERSISTENT menyimpan koneksi terbuka di worker PHP-FPM
- Koneksi yang tersimpan langsung dipakai ulang oleh request berikutnya
- Untuk skala cloud besar, dapat dipadukan dengan ProxySQL / PgBouncer

KONTROL KONEKSI MANUAL:

Laravel menyediakan method DB::reconnect() untuk menyegarkan koneksi dan DB::disconnect() untuk memutus koneksi secara eksplisit.

Konfigurasi Persistent PDO di Laravel:

```
// File: config/database.php
'mysql' => [
    'driver' => 'mysql',
    'host' => env('DB_HOST', '127.0.0.1'),
    'database' => env('DB_DATABASE', 'siakad'),
    'options' => [
        // Gunakan kembali koneksi di PHP-FPM:
        PDO::ATTR_PERSISTENT => true,
        PDO::ATTR_TIMEOUT => 5,
    ],
],
```

READ / WRITE CONNECTION SPLITTING

POIN PEMBAHASAN UTAMA:

- Trafik web umumnya didominasi operasi baca (80% SELECT, 20% WRITE)
- Beban database dibagi ke Server Primary (Write) & Server Replica (Read)
- Laravel mengarahkan query SELECT ke host 'read' secara otomatis
- Query INSERT, UPDATE, dan DELETE diarahkan otomatis ke host 'write'
- Opsi 'sticky' => true menjaga konsistensi baca sesaat setelah penulisan

TRANSPARAN BAGI DEVELOPER:

Kita tidak perlu mengubah kode Model atau Controller sedikit pun saat mengaktifkan arsitektur Read/Write Split di config/database.php.

Konfigurasi Read/Write di database.php:

```
'mysql' => [
    'driver' => 'mysql',
    'read' => [
        'host' => ['10.0.1.21', '10.0.1.22'],
    ],
    'write' => [
        'host' => ['10.0.1.10'], // Master DB
    ],
    'sticky' => true,
    'database' => 'siakad_del',
    'username' => env('DB_USERNAME'),
],
```

MULTI-DATABASE CONNECTIONS (HYBRID DBMS)

POIN PEMBAHASAN UTAMA:

- Satu aplikasi Laravel dapat terhubung ke banyak database sekaligus
- Contoh: Data Akademik di MySQL, sedangkan Data Log Audit di PostgreSQL
- Daftarkan profil koneksi tambahan pada array 'connections' di config
- Gunakan DB::connection('nama') saat memanggil Fluent Query Builder
- Atau definisikan properti protected $connection pada kelas Model

INTEGRASI LINTAS SISTEM KAMPUS:

Fitur ini sangat berguna saat membangun portal terpadu yang perlu mengambil data dari database SIAKAD, Perpustakaan, dan Keuangan.

Akses Multi-Database di Laravel 11:

```
// 1. Akses koneksi PostgreSQL via DB Facade
$logs = DB::connection('pgsql_log')
            ->table('security_logs')
            ->latest()
            ->limit(20)
            ->get();

// 2. Koneksi khusus pada Model Eloquent
class Book extends Model {
    protected $connection = 'mysql_perpus';
}
```

DATABASE TRANSACTIONS & CONCURRENCY LOCKING

POIN PEMBAHASAN UTAMA:

- DB::transaction() membungkus beberapa operasi tulis menjadi 1 kesatuan
- Jika seluruh baris kode sukses, COMMIT dijalankan secara otomatis
- Jika muncul Exception di tengah jalan, ROLLBACK otomatis dieksekusi
- Argumen kedua menentukan jumlah retry otomatis jika terjadi deadlock
- Method lockForUpdate() mencegah balapan akses (race condition) kuota

STUDI KASUS REBUTAN KUOTA KRS:

Dengan lockForUpdate(), database mengunci baris mata kuliah selama beberapa milidetik hingga pemotongan kuota mahasiswa pertama tuntas.

Implementasi DB::transaction & Locking:

```
DB::transaction(function () use ($nim, $mkId) {
    // Kunci baris hingga transaksi selesai
    $mk = Course::where('id', $mkId)
                ->lockForUpdate()
                ->firstOrFail();

    if ($mk->quota < 1) {
        throw new \Exception('Kuota Habis!');
    }
    $mk->decrement('quota');
    Enrollment::create([
        'course_id' => $mkId,
        'student_nim' => $nim,
    ]);
}, 3);
```

PENANGANAN ERROR KONEKSI (QUERYEXCEPTION)

POIN PEMBAHASAN UTAMA:

- Kesalahan koneksi atau pelanggaran constraint memicu QueryException
- Kelas ini mewarisi PDOException dan menyimpan kode standar SQLSTATE
- Gunakan blok try-catch untuk menangkap error database secara elegan
- Catat detail teknis ke log internal server dengan Log::error()
- Tampilkan pesan yang ramah dan aman kepada pengguna di halaman web

KEAMANAN PESAN ERROR:

Jangan pernah menampilkan isi pesan SQL mentah ke pengguna akhir karena dapat membocorkan nama tabel dan struktur kolom database Anda.

Menangkap QueryException dengan Aman:

```
use Illuminate\Database\QueryException;
use Illuminate\Support\Facades\Log;

try {
    Course::create($request->validated());
} catch (QueryException $e) {
    // Simpan log teknis untuk administrator
    Log::error('DB Error: ' . $e->getMessage());

    // Berikan pesan sopan ke pengguna
    return back()->withErrors([
        'msg' => 'Gagal menyimpan data ke DB.'
    ]);
}
```

PETA STRUKTUR DIREKTORI DATABASE LARAVEL 11

POIN PEMBAHASAN UTAMA:

- File .env: Menyimpan parameter kredensial koneksi database per mesin
- File config/database.php: Pusat registrasi seluruh driver & koneksi DB
- Folder database/migrations/: Menyimpan file skema tabel berurutan waktu
- Folder database/seeders/ & factories/: Populasi data awal & data dummy
- Folder app/Models/: Tempat seluruh kelas Model Eloquent ORM berada

ORGANISASI KODE BERSIH:

Struktur yang terstandarisasi memudahkan anggota tim baru langsung memahami letak konfigurasi, skema tabel, dan model data.

Struktur Folder Database Laravel 11:

```
laravel-app/
├── .env                        ◄── Kredensial DB
├── config/
│   └── database.php            ◄── Konfigurasi DB
├── app/
│   └── Models/
│       ├── Course.php          ◄── Model Eloquent
│       └── Enrollment.php
└── database/
    ├── factories/              ◄── Generator Dummy
    ├── migrations/             ◄── Skema Tabel
    └── seeders/                ◄── Data Awal
```

BEDAH FILE KONFIGURASI CONFIG/DATABASE.PHP

POIN PEMBAHASAN UTAMA:

- Bagian 'default': Menentukan nama koneksi utama yang dipakai aplikasi
- Bagian 'connections': Berisi daftar driver (sqlite, mysql, pgsql, sqlsrv)
- Charset 'utf8mb4': Menjamin kompatibilitas karakter unik & simbol
- Opsi 'strict' => true: Mencegah pemotongan data tidak valid di MySQL
- Bagian 'redis': Konfigurasi koneksi in-memory database untuk cache/sesi

CONFIG CACHING DI PRODUKSI:

Saat deployment, jalankan 'php artisan config:cache' agar pembacaan konfigurasi database berjalan instan dari memori.

Isi Kunci File config/database.php:

```
return [
    'default' => env('DB_CONNECTION', 'mysql'),

    'connections' => [
        'mysql' => [
            'driver' => 'mysql',
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE'),
            'username' => env('DB_USERNAME'),
            'password' => env('DB_PASSWORD'),
            'charset' => 'utf8mb4',
            'strict' => true,
        ],
    ],
];
```

STRUKTUR DATABASE MIGRATIONS (SCHEMA BUILDER)

POIN PEMBAHASAN UTAMA:

- Berfungsi sebagai Version Control untuk struktur tabel basis data
- Method up(): Mendefinisikan pembuatan tabel atau penambahan kolom baru
- Method down(): Membatalkan perubahan saat dilakukan migrate:rollback
- Menggunakan fasad Schema dan Blueprint untuk merancang tipe data kolom
- Dijalankan lewat perintah CLI: php artisan migrate

KOLABORASI TIM PENGEMBANG:

Dengan Migration, anggota tim cukup menjalankan 'git pull' lalu 'php artisan migrate' untuk menyamakan tabel di laptop masing-masing.

Struktur File Migration Laravel 11:

```
return new class extends Migration {
    public function up(): void {
        Schema::create('courses',
            function (Blueprint $table) {
                $table->id();
                $table->string('code', 10)->unique();
                $table->string('name', 100);
                $table->timestamps();
            }
        );
    }
    public function down(): void {
        Schema::dropIfExists('courses');
    }
};
```

STRUKTUR DATABASE SEEDERS & FACTORIES

POIN PEMBAHASAN UTAMA:

- Seeder bertugas mengisi data master awal (akun Admin, daftar Prodi)
- Factory bertugas membangkitkan data uji coba otomatis berbasis Faker
- Seluruh kelas seeder dikoordinasikan di dalam file DatabaseSeeder.php
- Sangat mempercepat pengujian aplikasi tanpa perlu input manual di form
- Perintah 'php artisan migrate:fresh --seed' mereset & mengisi ulang DB

EFISIENSI PENGUJIAN (QA):

Kita dapat membuat 1.000 baris data mahasiswa uji hanya dalam waktu kurang dari 2 detik menggunakan Model Factory.

Struktur DatabaseSeeder.php:

```
namespace Database\Seeders;
use Illuminate\Database\Seeder;
use App\Models\User;

class DatabaseSeeder extends Seeder {
    public function run(): void {
        // 1. Jalankan Seeder Mata Kuliah
        $this->call(CourseSeeder::class);

        // 2. Buat 50 Mahasiswa Uji via Factory
        User::factory()->count(50)->create();
    }
}
```

STRUKTUR DB FACADE & FLUENT QUERY BUILDER

POIN PEMBAHASAN UTAMA:

- Menggunakan fasad Illuminate\Support\Facades\DB untuk memulai query
- Menyediakan method chaining untuk SELECT, WHERE, JOIN, dan ORDER BY
- Mendukung fungsi agregasi cepat: count(), sum(), avg(), min(), max()
- Mengembalikan hasil berupa Collection yang berisi objek stdClass ringan
- Cocok untuk query laporan analitik dan penggabungan banyak tabel

PERFORMA TINGGI:

Karena tidak perlu menginstansiasi objek Model penuh, Query Builder mengonsumsi memori lebih kecil saat memproses puluhan ribu baris.

Query Join & Agregasi dengan DB Facade:

```
use Illuminate\Support\Facades\DB;

$rekap = DB::table('enrollments')
    ->join('courses',
           'enrollments.course_id', '=', 'courses.id')
    ->where('enrollments.semester', '2026-Ganjil')
    ->select(
        'enrollments.student_nim',
        DB::raw('SUM(courses.credits) as sks')
    )
    ->groupBy('enrollments.student_nim')
    ->get();
```

STRUKTUR ELOQUENT ORM & KONVENSI MODEL

POIN PEMBAHASAN UTAMA:

- Menerapkan Active Record Pattern: 1 kelas Model mewakili 1 tabel DB
- Nama Model tunggal (Course) otomatis memetakan tabel jamak (courses)
- Primary Key diasumsikan bernama 'id' dengan tipe auto-increment bigint
- Properti $fillable melindungi tabel dari serangan Mass Assignment
- Otomatis mengelola kolom waktu created_at dan updated_at

KEAMANAN MASS ASSIGNMENT:

Selalu definisikan properti $fillable pada setiap Model agar hanya kolom yang diizinkan saja yang dapat diisi dari input form pengguna.

Struktur Kelas Model Eloquent:

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Course extends Model {
    // Nama tabel di database
    protected $table = 'courses';

    // Kolom yang diizinkan diisi massal
    protected $fillable = [
        'code',
        'name',
        'credits',
        'quota',
    ];
}
```

RELASI ELOQUENT & OPTIMASI EAGER LOADING

POIN PEMBAHASAN UTAMA:

- Relasi antar tabel ditulis sebagai method: hasMany() dan belongsTo()
- Masalah klasik ORM adalah N+1 Query Problem akibat Lazy Loading di loop
- Menampilkan 30 mata kuliah beserta relasinya memicu 1 + 30 = 31 query
- Gunakan Eager Loading: with('enrollments') atau withCount('enrollments')
- Eager Loading memangkas puluhan query berulang menjadi hanya 2 query!

WAJIB UNTUK SKALABILITAS:

Membiarkan N+1 Query di aplikasi produksi akan menguras koneksi database dengan cepat saat diakses oleh banyak mahasiswa bersamaan.

Solusi N+1 dengan Eager Loading:

```
// HINDARI (Memicu N+1 Query di dalam loop):
$list = Course::all();
foreach ($list as $c) {
    echo $c->enrollments->count(); // Query terus!
}

// GUNAKAN EAGER COUNT (Hanya 1 Query SQL!):
$list = Course::withCount('enrollments')->get();
foreach ($list as $c) {
    echo $c->enrollments_count; // Cepat & hemat!
}
```

CONTOH KASUS: SISTEM KRS & MATA KULIAH IT DEL

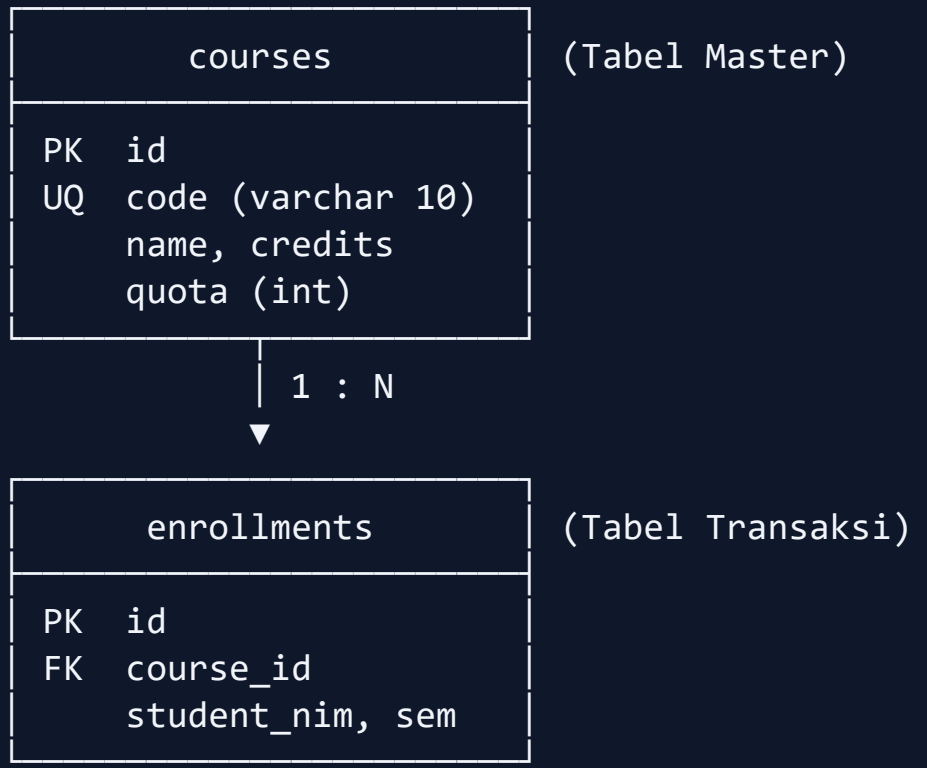
POIN PEMBAHASAN UTAMA:

- Deskripsi: Membangun modul Katalog Mata Kuliah dan Pengambilan KRS
- Tabel courses: Menyimpan kode unik, nama mata kuliah, SKS, dan kuota
- Tabel enrollments: Menyimpan relasi course_id, NIM mahasiswa, & semester
- Aturan Bisnis: Pengambilan KRS wajib mengurangi sisa kuota secara atomik
- Validasi: Mahasiswa tidak boleh mengambil mata kuliah jika kuota = 0

RELASI ONE-TO-MANY (1:N):

Satu baris pada tabel 'courses' berelasi dengan banyak baris pada tabel 'enrollments' melalui kunci tamu (foreign key) course_id.

Skema ERD Studi Kasus SIAKAD:



LANGKAH 1: KONFIGURASI .ENV & UJI KONEKSI DB

POIN PEMBAHASAN UTAMA:

- 1. Buat database baru bernama 'siakad_del' pada server MySQL lokal
- 2. Buka file .env proyek Laravel dan arahkan DB_DATABASE=siakad_del
- 3. Sesuaikan DB_USERNAME dan DB_PASSWORD sesuai akun MySQL lokal
- 4. Jalankan perintah 'php artisan db:show' pada terminal proyek
- 5. Pastikan status koneksi ke MySQL berhasil terverifikasi tanpa error

VERIFIKASI DINI:

Membiasakan uji koneksi dengan 'php artisan db:show' membantu mendeteksi salah password atau port MySQL yang belum aktif sejak awal.

Konfigurasi .env & Uji Artisan:

```
# 1. Konfigurasi di dalam file .env:
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=siakad_del
DB_USERNAME=root
DB_PASSWORD=

# 2. Jalankan verifikasi di Terminal:
$ php artisan db:show
  MySQL ............................. 8.0.36
  Database ......................... siakad_del
  Connection ........................ mysql
```

LANGKAH 2: MEMBUAT MIGRATION TABEL DATABASE

POIN PEMBAHASAN UTAMA:

- Jalankan perintah: php artisan make:migration create_siakad_tables
- Susun struktur kolom tabel 'courses' beserta indeks unique pada 'code'
- Susun struktur tabel 'enrollments' dengan foreignId('course_id')
- Pasang aturan unique(['course_id', 'student_nim']) cegah daftar ganda
- Jalankan 'php artisan migrate' untuk mengeksekusi pembuatan tabel

CASCADE ON DELETE:

Method cascadeOnDelete() memastikan jika suatu mata kuliah dihapus, maka data pendaftaran terkait ikut dibersihkan secara konsisten.

Isi File Migration create_siakad_tables:

```
Schema::create('courses', function ($t) {
    $t->id();
    $t->string('code', 10)->unique();
    $t->string('name', 100);
    $t->unsignedTinyInteger('credits');
    $t->unsignedInteger('quota')->default(40);
    $t->timestamps();
});
Schema::create('enrollments', function ($t) {
    $t->id();
    $t->foreignId('course_id')
      ->constrained()->cascadeOnDelete();
    $t->string('student_nim', 15);
    $t->unique(['course_id', 'student_nim']);
    $t->timestamps();
});
```

LANGKAH 3: MEMBUAT MODEL ELOQUENT & RELASI

POIN PEMBAHASAN UTAMA:

- Jalankan: php artisan make:model Course dan make:model Enrollment
- Tambahkan properti $fillable pada kedua kelas Model tersebut
- Definisikan method enrollments(): HasMany di dalam kelas Course
- Definisikan method course(): BelongsTo di dalam kelas Enrollment
- Kini kedua Model siap digunakan untuk operasi query dan relasi

KEMUDAHAN AKSES RELASI:

Setelah method relasi dibuat, kita dapat langsung memanggil $course->enrollments()->create(...) untuk menambah peserta mata kuliah.

Kode Model Course.php & Enrollment.php:

```
// File: app/Models/Course.php
class Course extends Model {
    protected $fillable = [
        'code', 'name', 'credits', 'quota'
    ];
    public function enrollments(): HasMany {
        return $this->hasMany(Enrollment::class);
    }
}

// File: app/Models/Enrollment.php
class Enrollment extends Model {
    protected $fillable = [
        'course_id', 'student_nim', 'semester'
    ];
}
```

LANGKAH 4: MEMBUAT SEEDER DATA MATA KULIAH

POIN PEMBAHASAN UTAMA:

- Jalankan perintah CLI: php artisan make:seeder CourseSeeder
- Siapkan data mata kuliah IT Del di dalam method run()
- Gunakan Course::updateOrCreate() berdasarkan kunci unik 'code'
- Eksekusi seeder: php artisan db:seed --class=CourseSeeder
- Data awal mata kuliah kini telah siap di dalam tabel MySQL

MENGAPA UPDATEORCREATE?

Method updateOrCreate() mencegah duplikasi data apabila perintah db:seed dijalankan lebih dari satu kali oleh developer.

Kode File CourseSeeder.php:

```
class CourseSeeder extends Seeder {
    public function run(): void {
        $data = [
            ['code' => '12S3101', 'credits' => 4,
             'name' => 'PPW', 'quota' => 45],
            ['code' => '12S2203', 'credits' => 3,
             'name' => 'Basis Data', 'quota' => 40],
        ];
        foreach ($data as $row) {
            Course::updateOrCreate(
                ['code' => $row['code']],
                $row
            );
        }
    }
}
```

LANGKAH 5: CONTROLLER READ, FILTER, & PAGINATION

POIN PEMBAHASAN UTAMA:

- Buat controller dengan: php artisan make:controller CourseController
- Gunakan Course::withCount('enrollments') untuk mengambil jumlah peserta
- Tambahkan filter pencarian dinamis menggunakan method when()
- Urutkan daftar mata kuliah berdasarkan kolom kode secara ascending
- Batasi tampilan 10 data per halaman dengan method paginate(10)

QUERY RINGKAS & OPTIMAL:

Seluruh proses hitung relasi, filter pencarian, pengurutan, dan paginasi dieksekusi secara efisien di sisi database server.

Method index() di CourseController.php:

```
public function index(Request $req) {
    $courses = Course::withCount('enrollments')
        ->when($req->search, function ($q, $s) {
            $q->where('code', 'like', "%{$s}%")
              ->orWhere('name', 'like', "%{$s}%");
        })
        ->orderBy('code')
        ->paginate(10)
        ->withQueryString();

    return view('courses.index', [
        'courses' => $courses
    ]);
}
```

LANGKAH 6: CONTROLLER TRANSAKSI PENGAMBILAN KRS

POIN PEMBAHASAN UTAMA:

- Validasi input student_nim yang dikirim dari form pendaftaran
- Bungkus proses di dalam DB::transaction() demi jaminan Atomicity
- Ambil data mata kuliah menggunakan lockForUpdate() agar aman
- Periksa apakah sisa kuota masih tersedia (> 0) sebelum memotong
- Kurangi kuota dan simpan data Enrollment, lalu kirim pesan sukses

KEANDALAN TRANSAKSI:

Jika mahasiswa yang sama mencoba mendaftar dua kali, constraint unique akan melempar error dan otomatis membatalkan pemotongan kuota.

Method enroll() di CourseController.php:

```
public function enroll(Request $req, int $id) {
    $req->validate(['nim' => 'required|string']);
    DB::transaction(function () use ($req, $id) {
        $mk = Course::where('id', $id)
                    ->lockForUpdate()
                    ->firstOrFail();
        if ($mk->quota < 1) {
            abort(422, 'Kuota kelas penuh!');
        }
        $mk->decrement('quota');
        $mk->enrollments()->create([
            'student_nim' => $req->nim,
            'semester' => '2026-Ganjil',
        ]);
    });
    return back()->with('ok', 'KRS Berhasil!');
}
```

LANGKAH 7: INTEGRASI ROUTES & TAMPILAN BLADE

POIN PEMBAHASAN UTAMA:

- Daftarkan route GET dan POST pada file routes/web.php
- Tampilkan daftar mata kuliah menggunakan perulangan @foreach di Blade
- Cetak atribut {{ $c->enrollments_count }} hasil perhitungan withCount
- Sertakan direktif keamanan @csrf di dalam form pengiriman NIM
- Tampilkan tombol halaman dengan memanggil {{ $courses->links() }}

SIKLUS MVC + DATABASE LENGKAP:

Pengguna kini dapat mencari mata kuliah, melihat sisa kuota real-time, dan mendaftar KRS dengan jaminan keamanan transaksi database.

Kode routes/web.php & View Blade:

```
// 1. File: routes/web.php
Route::get('/krs', [CourseCtrl::class, 'index']);
Route::post('/krs/{id}', [CourseCtrl::class, 'save']);

// 2. File: views/courses/index.blade.php
@foreach ($courses as $c)
  <tr>
    <td>{{ $c->code }} (Kuota: {{ $c->quota }})</td>
    <td>
      <form method="POST" action="/krs/{{ $c->id }}">
        @csrf <input name="nim">
        <button type="submit">Ambil</button>
      </form>
    </td>
  </tr>
@endforeach
```

BEST PRACTICES & TUGAS PRAKTIKUM MINGGU 6

POIN PEMBAHASAN UTAMA:

- Selalu simpan kredensial di .env dan jangan pernah di-hardcode
- Gunakan Parameter Binding / Eloquent untuk mencegah SQL Injection
- Gunakan Eager Loading (with / withCount) untuk menghindari N+1 Query
- Tugas 1: Buat fitur 'Batal KRS' yang mengembalikan kuota via transaksi
- Tugas 2: Buat halaman detail mata kuliah berisi tabel NIM pendaftar

TARGET CAPAIAN PRAKTIKUM:

Mahasiswa mendemonstrasikan fitur Ambil & Batal KRS yang terhubung ke MySQL dengan skema Migration dan pengujian transaksi.

Checklist Pengujian Praktikum:

```
[✓] Koneksi MySQL aktif ('php artisan db:show')
[✓] Tabel terbentuk via 'php artisan migrate'
[✓] Data awal terisi via 'php artisan db:seed'
[✓] Pencarian & Paginasi berjalan tanpa N+1
[✓] Ambil & Batal KRS memakai DB::transaction()
[✓] Penanganan QueryException berjalan rapi
```

REFERENSI RESMI & SESI DISKUSI TANYA JAWAB

POIN PEMBAHASAN UTAMA:

- Laravel 11 Docs: Database Getting Started, Configuration, & Read/Write
- Laravel 11 Docs: Migrations, Seeding, Query Builder, & Eloquent ORM
- PHP Manual: PHP Data Objects (PDO) & Prepared Statements Security
- Martin Fowler: Patterns of Enterprise Application Architecture
- Dosen Pengampu: Chandro Pardede, S.Kom., M.Sc. • FITE IT Del

PESAN PENUTUP MINGGU 6:

'Aplikasi yang tangguh berdiri di atas konektivitas data yang aman, skema yang terversi rapi, dan transaksi yang menjaga integritas.'

Referensi Dokumentasi & Kontak:

[1] Dokumentasi Database Laravel 11:
https://laravel.com/docs/11.x/database

[2] Dokumentasi Eloquent ORM:
https://laravel.com/docs/11.x/eloquent

[3] OWASP SQL Injection Prevention:
https://cheatsheetseries.owasp.org/

Terima Kasih! Silakan ajukan pertanyaan.